MST Lab – Minimum Spanning Tree Visualiser

An interactive graph algorithm learning tool designed to transform how students understand one of computer science's most fundamental concepts.

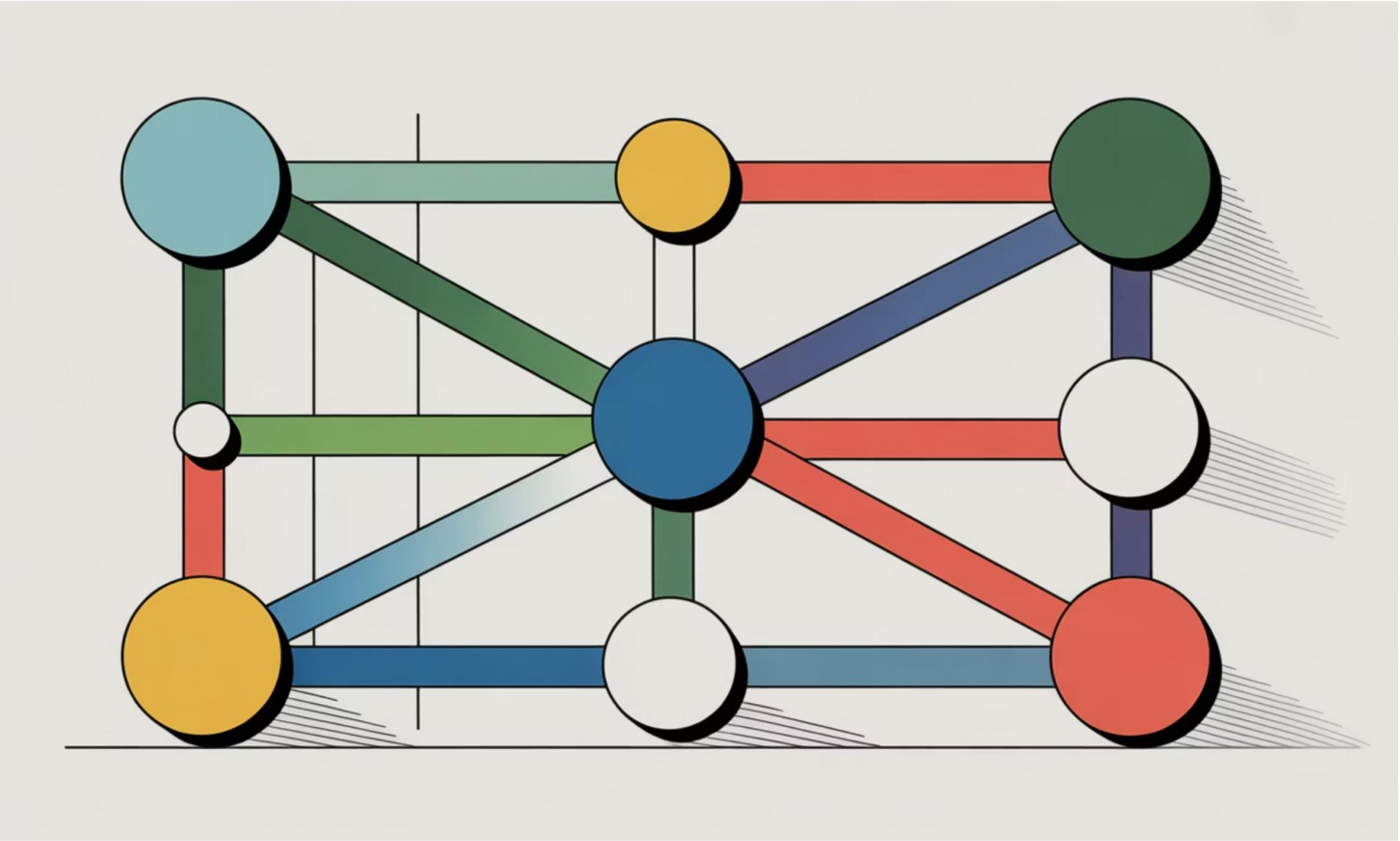
Presented by: *Dhruv Faldu*

Understanding Minimum Spanning Trees

What is an MST?

A Minimum Spanning Tree is a subset of edges in a connected, weighted, undirected graph that connects all vertices together whilst minimising the total edge weight. Crucially, it contains no cycles and touches every vertex exactly once.

Think of it as finding the most efficient way to connect everything—like planning the shortest road network or laying cables with minimal material cost.



Why MST Matters in Real-World Applications

Network Design

Planning efficient road systems, electrical grids, and pipeline networks that minimise construction costs whilst ensuring all locations are connected.

Computer Networks

Optimising routing protocols and determining the most efficient paths for data transmission across interconnected devices.

Cost Optimisation

Solving complex optimisation problems in logistics, telecommunications, and infrastructure planning where efficiency directly translates to savings.

Graph Theory Foundation

Building essential understanding of graph algorithms that form the basis for more advanced computational problems and research.

Project Overview: MST Lab

*MST Lab is an **interactive web-based visualisation tool** that brings graph algorithms to life. Users can construct custom graphs, generate random networks, and watch MST algorithms execute in real-time with clear visual feedback.*

1

Build Your Graph

Create vertices and edges manually using adjacency matrix input, or let the system generate random graphs automatically for experimentation.

2

Visualise the Process

Watch as the graph renders on an interactive canvas, with edges highlighted and weights displayed clearly for easy understanding.

3

Execute & Learn

Run Prim's or Kruskal's algorithm step-by-step, observing how each decision builds the minimum spanning tree incrementally.

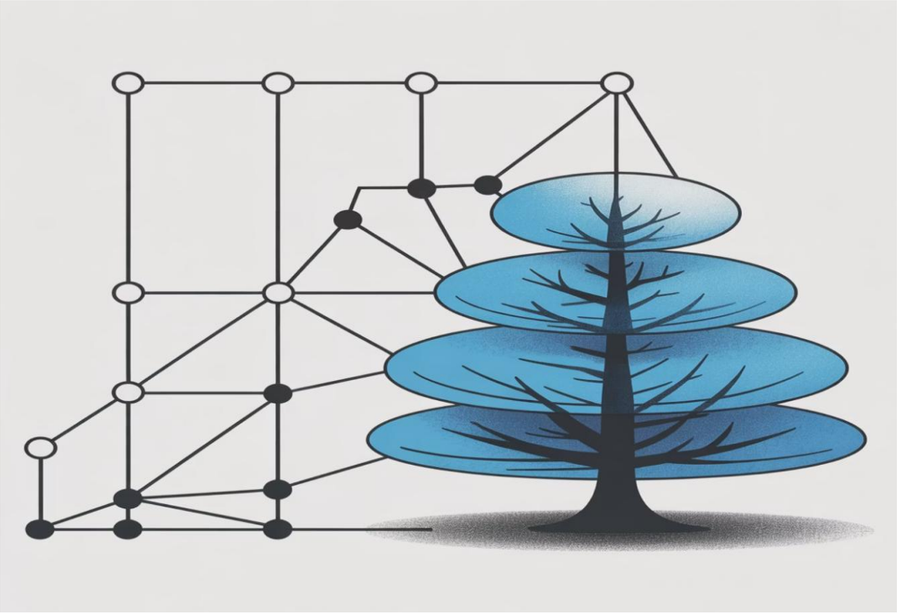
4

Analyse Results

View the final MST with total weight calculation, edge selection sequence, and performance metrics displayed clearly.

Two Powerful Algorithms Compared

Prim's Algorithm



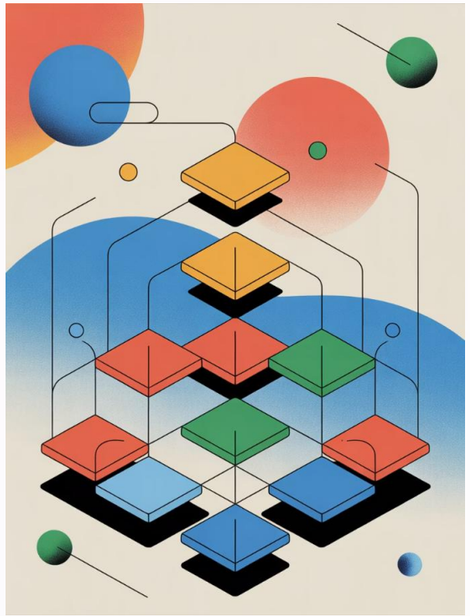
Approach: *Greedy expansion from a single starting node*

How it works: *Begins at one vertex and progressively adds the minimum-weight edge that connects a new vertex to the growing tree. This continues until all vertices are included.*

Best for: *Dense graphs with many edges*

Time complexity: *$O(V^2)$ with adjacency matrix, O(E log V) with priority queue*

Kruskal's Algorithm



Approach: *Global edge selection with cycle detection*

How it works: *Sorts all edges by weight first, then adds edges one by one, skipping those that would create cycles. Uses Union-Find data structure for efficient cycle detection.*

Best for: *Sparse graphs with fewer edges*

Time complexity: *O(E log E) due to sorting step*

Both algorithms guarantee finding the minimum spanning tree, but their different approaches make them suitable for different scenarios. MST Lab lets you compare their execution side-by-side!

Looking Ahead: Future Enhancements

01

Enhanced Animation Controls

Add play, pause, step forward, and step backward functionality to give users complete control over algorithm execution speed and progression.

02

Large Graph Support

Optimise performance to handle graphs with hundreds or thousands of vertices, expanding the tool's applicability to real-world problems.

03

Visual Highlighting

Implement dynamic edge colouring and animation during algorithm execution to clearly show which edges are being considered and selected at each step.

04

Performance Comparison

Provide side-by-side execution time metrics and step counts for both algorithms, helping students understand practical efficiency differences.

05

Export Capabilities

Allow users to save visualisations as images, export results as reports, or share custom graphs with classmates and instructors.

Thank you for exploring MST Lab! *This visualiser transforms abstract graph theory into tangible, interactive learning experiences.* 🙏